Five Methodological Tiers of Entity Resolution: A Comparative Study Across Four Benchmark Datasets

Johnny Saldana[1]

[1]Department of Computer Science, Johns Hopkins University , jsaldan2@jhu.edu

EN.601.507 Applied Entity Resolution and Deduplication
Spring 2023
Advised by Dr. Tom Lippincott, Center for Language and Speech Processing

Abstract

Entity resolution is the problem of deciding whether two records refer to the same real-world entity. It sits at the foundation of data integration, master-data management, and any analytical workflow that draws on more than one source. We present `matchify`, a reproducible Python package that implements five entity-resolution models spanning the methodological progression of the field, from a hash-based exact-match baseline to a fine-tuned twin-encoder Siamese network. The five models are exposed under a single abstract interface and evaluated on four benchmark datasets (Amazon-Google Products, Abt-Buy, DBLP-ACM, and a controlled synthetic person-record corpus). We adopt a held-out group split protocol so that supervised models never observe the group identifiers used at evaluation time, and we report mean reciprocal rank and threshold-swept precision-recall curves with multi-seed standard deviations on the stochastic models. The empirical findings are consistent with the literature in three of four configurations and surprising in one. On the cross-catalog Abt-Buy benchmark a supervised multilayer perceptron over engineered field similarities outperforms both pretrained sentence-BERT and a fine-tuned Siamese variant by a wide margin, suggesting that on small-vocabulary, high-variance product corpora, the choice of feature representation still dominates the choice of architecture.

1 Introduction

Most modern data integration pipelines combine sources that lack unique entity identifiers (Christen, 2012). The result is a record linkage problem at scale. Every workflow that combines census tables, hospital records, customer databases, or e-commerce catalogs must decide which records refer to the same real-world entity. The problem appears under several names in the literature, including record linkage, deduplication, object identification, and entity matching. We use the term entity resolution following the survey of Papadakis et al. (2021).

The field has been studied for more than half a century, beginning with the probabilistic record linkage formalism of Fellegi and Sunter (1969). It has since absorbed advances from string similarity, machine learning, distributed systems, crowd computation, and most recently deep learning. Christen (2012) provides the canonical textbook treatment. Papadakis et al. (2021) organize the entire history into four generations, characterized by the dominant paradigm of each era. Binette and Steorts (2022) review the modern probabilistic and Bayesian methods that have grown out of the Fellegi-Sunter framework.

Despite this history the field continues to evolve. Mudgal et al. (2018) demonstrate that a deep learning architecture they call DeepMatcher outperforms classical rule-based systems on a range of benchmarks. Ebraheem et al. (2018) introduce DeepER, a deep representation learning approach to entity resolution built on distributed tuple representations. Li et al. (2020) extend the deep line by fine-tuning pretrained transformer language models, achieving state-of-the-art F1 on every standard benchmark. Yet the underlying problem has not changed, and the choice of method for any given dataset remains a question of judgment.

This paper presents `matchify`, a Python package that implements five entity resolution models under a single abstract interface and evaluates them on four benchmark datasets under a consistent experimental protocol. The five models trace the methodological progression of the field. The first is a hash-based exact-match baseline that returns the rules-of-the-game lower bound. The second is a configurable rule-based field-similarity model in the spirit of the Magellan toolkit (Konda et al., 2016), supporting per-field type-aware normalization, four string-similarity metrics, and four blocking strategies. The third is a supervised multilayer perceptron trained on sampled positive and negative pairs, with each pair represented as a fixed-length vector of per-field similarity scores. The fourth uses a pretrained sentence-transformer (Reimers and Gurevych, 2019) without fine-tuning, ranking candidate records by cosine similarity in the encoder's output space. The fifth fine-tunes the same encoder under a contrastive objective (Hadsell et al., 2006). The twin-encoder Siamese configuration for entity resolution is studied in detail by Ebraheem et al. (2018), who introduce a related deep representation learning approach called DeepER.

The contribution of this paper is not a novel method. Each of the five models is well-grounded in prior work. The contribution is a reproducible reference implementation under a single interface, a disciplined evaluation protocol that holds out groups rather than individual records, and a discussion of where the empirical results align with the expectations the literature would set and where they diverge. The full code, data, and trained model artifacts are available at the package repository.

The remainder of the paper is organized as follows. Section 2 surveys the relevant background and prior work. Section 3 describes the five models in detail, with citations to the methodological line each draws on. Section 4 introduces the four benchmark datasets. Section 5 specifies the evaluation protocol. Section 6 reports the results. Section 7 discusses where the findings align with expectations and where they do not. Section 8 compares against published F1 numbers from DeepMatcher and Ditto. Section 9 acknowledges limitations. Section 10 documents reproducibility.

2 Background and Related Work

2.1 Definitions

Following Christen (2012), let A and B be two databases of records described over the same or partially overlapping schemas. The entity resolution task is to determine, for every pair of records (a, b) with a in A and b in B, whether the two records refer to the same real-world entity. A binary decision suffices for many applications, but more general formulations (Binette and Steorts, 2022) ask for a clustering of all records, a probability of being a match, or a ranked list of candidate matches per query record. The package described here implements the ranked-list formulation primarily and derives binary classification at any chosen threshold from those rankings.

2.2 Generations of entity resolution

Papadakis et al. (2021) organize the field into four generations. The first is the probabilistic linkage tradition of Fellegi and Sunter (1969), in which the match decision is grounded in a likelihood ratio of the observed pair under a match hypothesis versus a non-match hypothesis. The second generation introduces approximate string similarity and rule-based decision systems. Bilenko and Mooney (2003) study string-similarity functions adapted to the matching task. Cohen et al. (2003) provide a comparative empirical study of string distances. Benjelloun et al. (2009) generalize rule-based matching to a generic algebraic framework called Swoosh, treating match and merge functions as black boxes whose properties (idempotence, commutativity, associativity, representativity) determine which efficient algorithm applies. The third generation incorporates supervised machine learning, blocking, and parallel execution. Konda et al. (2016) describe Magellan, a toolkit that supports blocking, sampling, labeling, supervised pair classification, and quality debugging. Wang et al. (2012) introduce CrowdER, which delegates the most uncertain pairs to crowd workers under a hybrid human-in-the-loop scheme. Bahmani et al. (2017) describe ERBlox, a hybrid that combines matching dependencies with classifier-based duplicate detection. The fourth generation is the deep learning era. Mudgal et al. (2018) introduce DeepMatcher and study a design space of attribute encoders, attention mechanisms, and hybrid combinations. Ebraheem et al. (2018) introduce DeepER, which uses distributed tuple representations built on word embeddings combined with LSTM-based similarity composition. Li et al. (2020) extend the line with Ditto, a fine-tuned transformer that establishes new state of the art across multiple benchmarks. The five models in `matchify` correspond, roughly, to generations one through four of

this taxonomy.

2.3 Blocking

For a database of N records, a naive comparison of every pair scales quadratically and is intractable beyond a few thousand records. The standard remedy is blocking, also called indexing, in which a cheap keying function partitions the database so that comparisons need only be performed within blocks. Christen (2012) catalogs the standard blocking strategies, including standard equality blocking, sorted neighborhood, q-gram blocking, and canopy clustering. The package supports prefix blocking, sorted neighborhood, equality blocking, and a no-block "full" mode for small datasets. Blocking choice is part of the model configuration in all five tiers.

2.4 Evaluation

Papadakis et al. (2021) note that ER evaluation is conducted with two families of metrics. The classification family includes precision, recall, and F1 over the set of all candidate pairs that the model produces. The ranking family includes mean reciprocal rank (MRR), mean average precision, and recall at k. A practitioner who needs a final yes-or-no decision will typically use F1 at a chosen threshold. A practitioner who needs ranked candidate lists for downstream review will use MRR. The evaluation in this paper reports both, with the recognition that the two metrics measure different things.

The protocol caveat that we discuss in Section 8 is that classification F1 reported in the deep ER literature (e.g., Mudgal et al., 2018; Li et al., 2020) is computed on a labeled-pair test set, that is, a set of pairs sampled from the perfect mapping, and not on the all-pairs candidate set produced by blocking. This makes their numbers an upper bound on what is achievable rather than a direct head-to-head comparison.

3 Models

3.1 ExactMatchModel

The first model is a hash-based exact-match baseline. For each record r in the input table, we compute the hash of the tuple of all non-ignored fields. Two records are predicted to refer to the same entity if and only if they have the same hash. There is no training phase, no blocking step, and no notion of partial similarity.

This baseline is included for two reasons. The first is to establish a lower bound on what any nontrivial method should beat. The second is to surface datasets in which exact duplication is sufficient. Section 6 shows that on the controlled synthetic person dataset, where 20% of the records are perfect duplicates by construction, the exact-match baseline achieves recall 0.41 and precision 1.0 at threshold 0.5.

3.2 FlexMatchModel

The second model is a configurable rule-based field-similarity model in the spirit of Magellan (Konda et al., 2016). For each pair of records the model computes a normalized similarity score per field, sums those scores into a single record-level score, and ranks candidates by that score. Each field is associated with a type that selects an appropriate normalizer, drawing on the type-aware preprocessing strategies catalogued by Christen (2012, chap. 3). Names are parsed and lowercased. Phone numbers are normalized to E.164 format with a US default region. Addresses are parsed and lowercased. Dates are parsed and serialized as ISO 8601. Free text receives no type-specific normalization. Each field is also associated with a similarity metric. The package supports Jaro-Winkler (Winkler, 1990), Levenshtein (Levenshtein, 1966), TF-IDF cosine, and Jaccard token overlap. These four are the standard set studied in Bilenko and Mooney (2003) and Cohen et al. (2003).

The model fits a TF-IDF vectorizer on the corpus during the train phase. At inference time, blocking selects a candidate set, the configured similarity functions are evaluated for every (query, candidate) pair, the per-field similarities are summed, and candidates are returned in descending score order. The model is deterministic given a fixed configuration. No supervised signal is used.

3.3 MLPMatchModel

The third model lifts the rule-based field similarity into a learned weighting. For each candidate pair the model computes a fixed-length feature vector consisting of the four similarity metrics applied to each of the configured fields. With four fields and four metrics this yields a sixteen-dimensional vector per pair. A

multilayer perceptron classifier (Rumelhart et al., 1986) is trained on a sample of labeled pairs drawn from the `group_id` supervision, with positive pairs sharing a `group_id` and negative pairs not. The default sample is 4000 pairs, evenly split between positive and negative. At inference time the trained classifier produces a match probability for each candidate pair, and that probability is the ranking score.

This model corresponds to the supervised pairwise approach of Magellan (Konda et al., 2016) and to the hybrid in Bahmani et al. (2017) that combines declarative matching rules with a learned classifier. The contribution of the MLP over a hand-tuned weighted sum of similarities is that the model learns which features correlate with match identity on the specific dataset being processed. This is particularly relevant when the field similarities exhibit non-monotone interactions, for example when a near-perfect title match counts more than a perfect manufacturer match on a product-catalog dataset.

3.4 BertMatchModel

The fourth model embeds each record into a dense vector using a pretrained sentence-transformer (Reimers and Gurevych, 2019). The default backbone is `all-MiniLM-L6-v2`, a six-layer MiniLM-style transformer with a 384-dimensional output. Each record is serialized into a single string by concatenating its configured fields with a field-name prefix and a pipe separator, and the full table is encoded once in a batched forward pass. Inference compares a query record's embedding against all candidate embeddings (after blocking) using cosine similarity in the L2-normalized space.

The crucial property of this model is that no fine-tuning occurs. The encoder is used as a generic semantic similarity function, drawing on the broad pretraining corpus of sentence-transformers. The model therefore captures linguistic similarity that string-distance functions cannot, for example recognizing that "Sony Wireless Speaker" and "Bluetooth Speaker by Sony" are likely the same product family. But it also brings in pretraining biases that may not match the dataset's actual entity definitions.

This model is the analogue of the inference-time application of distributed tuple representations that Ebraheem et al. (2018) study, but without their training procedure.

3.5 SiameseMatchModel

The fifth model fine-tunes the same sentence-transformer encoder on labeled positive and negative pairs sampled from the `group_id` supervision. The training objective is the contrastive loss of Hadsell et al. (2006), which pulls the embeddings of positive pairs together and pushes the embeddings of negative pairs apart with a margin parameter. The fine-tuned encoder is then used identically to the BertMatchModel at inference time, and candidate ranking is again by cosine similarity in the encoder's output space.

The architecture is the canonical twin-encoder Siamese network in which the same encoder is applied to both records of a pair. Twin encoders for entity resolution are studied by Ebraheem et al. (2018), who introduce DeepER, an LSTM-based deep model over distributed tuple representations. The contribution of fine-tuning over the pretrained encoder is that the embedding geometry adapts to the specific entity definitions of the dataset. Pretrained semantic similarity is replaced by dataset-specific match similarity. Section 7 discusses when this adaptation helps and when it does not.

The default training schedule is 1 epoch over 4000 sampled pairs at batch size 32, with linear warmup over 10% of the training steps. The optimizer is the AdamW default of the sentence-transformers library (Loshchilov and Hutter, 2019).

3.6 Common interface

All five models inherit from an abstract base class `ERBaseModel` that specifies the methods `train(...)`, `predict(record, ...)`, `mrr()`, `confusion_matrix(threshold)`, and `threshold_sweep()`. The latter three implement the evaluation logic uniformly, walking the held-out test rows once and computing both ranking and classification metrics from the same set of predicted candidate scores. This uniform interface ensures that all five models are evaluated under exactly the same protocol, with the same blocking, the same candidate set, and the same scoring rules.

4 Datasets

The package bundles four datasets. Three are real-world benchmarks from the Leipzig DB-Group benchmark collection. The fourth is a synthetic person-record corpus generated by the package itself.

Amazon-Google Products contains approximately 3700 product records drawn from two e-commerce catalogs, with 1300 hand-curated matched pairs. The fields are name, description, manufacturer, and price. The two catalogs use different naming conventions (for example "iPod 30GB" versus "Apple iPod 30 GB Black"), so the dataset rewards methods that can tolerate moderate string variation.

Abt-Buy is a smaller e-commerce dataset, with about 2200 records and 1100 matched pairs across the Abt and Buy.com catalogs. The two catalogs differ more aggressively in their product descriptions than Amazon and Google do, with Abt offering long marketing prose and Buy offering terse listings. Section 7 discusses the consequences for model selection.

DBLP-ACM is a bibliographic benchmark drawn from two computer science publication databases, with about 5000 records and 2200 matched pairs. The fields are title, authors, venue, and year. Titles are nearly identical across the two sources for matching publications, so the dataset is widely viewed as a near-saturated benchmark on which all reasonable methods perform well.

Synthetic People is a controlled benchmark generated by the package's `person_faker` utility. The default 500-record corpus contains roughly 50% unique singletons, 20% exact duplicates of unique records, 20% close-match perturbations of unique records, and 10% close-non-match perturbations that do not correspond to any other record. The fields are first name, last name, birthdate, address, and phone number, all of which exercise the type-aware normalizers in the package's base class. The synthetic corpus serves as a controlled setting in which the data-generating process is fully known.

5 Experimental Protocol

We adopt a held-out group split. For each dataset we partition the unique `group_id` values 70/30, with the smaller partition reserved for evaluation. Every record whose `group_id` is in the test partition is a test row. Records whose `group_id` is in the train partition, or which have a NaN or singleton `group_id`, become train rows. The split is deterministic given the random-state parameter.

For the supervised models (MLP and Siamese) the labeled pair sampling draws only from train rows. The encoder fine-tuning step in the Siamese model and the classifier fitting step in the MLP model never observe a record whose `group_id` is in the test partition. This is the standard protocol for honest evaluation of supervised entity resolution models, but as Mudgal et al. (2018) note, it has not always been followed in the literature.

At evaluation time all five models produce, for each test row, a ranked list of candidate matches drawn from the full corpus including both train and test rows. This matches the deployed setting in which a query record is ranked against an existing record store.

We report two metrics. Mean reciprocal rank (MRR) is computed by locating the first true positive in each ranked candidate list and averaging the reciprocal of its rank across all test rows. The classification metrics, precision, recall, and F1, are reported at a single fixed threshold of 0.5 over all candidate pairs the model produced. We additionally report a precision-recall curve produced by sweeping the threshold over 51 evenly-spaced points in [0, 1].

For the stochastic models (MLP and Siamese), we run training and evaluation with three random seeds (`random_state` 0, 1, 2) and report the mean and sample standard deviation of MRR and F1. The deterministic models (Exact, Flex, BERT) ignore the seed parameter and return a single value.

The first 500 records of each benchmark are used. This is a slice chosen to fit a single-laptop runtime budget, and Section 9 discusses the implications.

6 Results

Tables 1, 2, 3, and 4 report MRR, precision, recall, and F1 for each of the five models on each of the four datasets. Stochastic models are reported as mean ± stddev across three seeds.

Precision-recall curves swept across 51 evenly-spaced thresholds in [0, 1] are shown in Figure 1. The curves are useful for inspecting how each model trades precision against recall, since the single-threshold F1 reports do not capture the shape of that trade.

Table 1: Amazon-Google Products. 500 records, 142 test rows.

Model	MRR	Prec	Rec	F1
ExactMatch	0.239	1.000	0.200	0.333
FlexMatch	0.465	0.095	0.881	0.172
MLP	0.602 ± 0.048	0.277	0.986	**0.431** ± 0.057
Bert	0.639	0.127	0.990	0.224
Siamese	**0.667** ± 0.019	0.095	1.000	0.174 ± 0.008

Table 2: Abt-Buy. 500 records, 151 test rows.

Model	MRR	Prec	Rec	F1
ExactMatch	0.000	0.000	0.000	0.000
FlexMatch	0.169	0.022	0.303	0.042
MLP	**0.714** ± 0.078	0.179	0.967	**0.302** ± 0.024
Bert	0.516	0.063	1.000	0.118
Siamese	0.638 ± 0.073	0.044	1.000	0.085 ± 0.018

7 Discussion

The five-model design lets us examine where the methodological progression of the field actually pays off and where it does not. Four datasets surface four distinct patterns.

7.1 Loose-text product catalogs reward fine-tuning (Amazon-Google)

On Amazon-Google the MRR improves monotonically with model sophistication. ExactMatch produces 0.24, the rule-based Flex produces 0.47, the supervised MLP produces 0.60, the pretrained BERT produces 0.64, and the fine-tuned Siamese produces 0.67. This is the pattern the literature would predict for a free-text dataset in which the same product is described in materially different language by the two sources. The fine-tuned encoder benefits from having seen positive and negative pairs from the dataset itself, and the resulting embedding geometry rewards near-paraphrases of product descriptions even when the surface forms diverge. The per-seed variance on Siamese (0.019) is small enough that the four-MRR-point lead over BERT is unlikely to be seed noise.

7.2 Cross-catalog text breaks BERT but not MLP (Abt-Buy)

The Abt-Buy result is the surprising one. The MLP achieves MRR 0.71, significantly ahead of pretrained BERT at 0.52 and of the fine-tuned Siamese at 0.64. This inverts the order on Amazon-Google. Three observations help explain it. First, Abt-Buy is the smallest of the real-world benchmarks (about 2200 records) and the supervision signal is correspondingly thinner. The seed standard deviations for MLP (0.078) and Siamese (0.073) are both more than three times what they were on Amazon-Google, suggesting that pair sampling matters more when the underlying training set is smaller. Second, Abt and Buy describe products in radically different registers, with Abt offering long marketing-prose descriptions and Buy offering terse listings. This kind of cross-domain language gap is exactly the case that generic semantic similarity, as offered by the pretrained sentence-transformer, fails to bridge. The MLP, by contrast, learns on the labeled pairs which similarity dimensions actually carry signal, and it can downweight the prose-versus-terse mismatch in favor of correlations on name and price. Third, the Siamese model does help over pretrained BERT (0.64 versus 0.52) but does not catch up to MLP. We interpret this as evidence that on a small, noisy benchmark, the encoder fine-tuning has too little supervision to specialize the embedding geometry usefully.

The takeaway from Abt-Buy is that on this kind of cross-catalog small-vocabulary corpus, engineered features plus a learned weighting still beat pretrained encoders, with or without fine-tuning. This is consistent with observations in the older line of supervised ER research (Bilenko and Mooney, 2003; Cohen et al., 2003) but is not the conclusion a casual reading of the deep-ER literature would predict.

Table 3: DBLP-ACM. 500 records, 150 test rows.

Model	MRR	Prec	Rec	F1
ExactMatch	0.000	0.000	0.000	0.000
FlexMatch	0.933	0.306	1.000	0.469
MLP	0.920 ± 0.013	**1.000**	**1.000**	**1.000**
Bert	0.933	0.664	1.000	0.798
Siamese	0.920 ± 0.013	0.491	1.000	0.659 ± 0.019

Table 4: Synthetic People. 500 records, 105 test rows.

Model	MRR	Prec	Rec	F1
ExactMatch	0.476	1.000	0.411	0.583
FlexMatch	0.929	0.307	1.000	0.470
MLP	0.919 ± 0.027	0.909	1.000	**0.952** ± 0.012
Bert	**0.933**	0.197	1.000	0.329
Siamese	0.917 ± 0.021	0.608	1.000	0.756 ± 0.028

7.3 Saturated structured benchmark (DBLP-ACM)

DBLP-ACM is widely understood to be near-saturated. The four trained models all reach MRR 0.92 to 0.93, and the rank ordering they impose on candidate matches is essentially identical. The classification F1 at the fixed 0.5 threshold separates the models, with the MLP achieving perfect P=R=F1=1.0 and the cosine-similarity models trailing because their score distributions place too many non-matching pairs above 0.5 at inference time. This is not a failure of ranking but of threshold calibration.

Ditto (Li et al., 2020) reports F1 0.989 on this dataset using its labeled-pair protocol. The MLP's perfect F1 in our setting reflects both the saturated nature of DBLP-ACM and our use of a calibrated classifier head as the score, which yields a usefully bimodal score distribution at inference time.

7.4 Synthetic perturbation favors engineered features (Synthetic People)

The synthetic person corpus poses a different challenge. Close-match pairs are produced by random one-character or one-field perturbations that do not follow any systematic pattern. There is no semantic relationship between a name with a leading character dropped and the original name, in the sense that a pretrained language model would have learned. The MLP wins F1 by a wide margin (0.95 versus 0.76 for Siamese, 0.47 for Flex, and 0.33 for BERT). This is again consistent with the Abt-Buy story. When the data-generating process is character-noise rather than semantic-paraphrase, the signal lives in specific string-similarity scores rather than in a learned embedding geometry.

The MRR numbers on Synthetic People are also notably higher than they would be on a full-data evaluation. This is because the held-out test set excludes singletons, which by construction have no positive match to find in the corpus, and which would otherwise dilute the MRR.

7.5 Synthesis

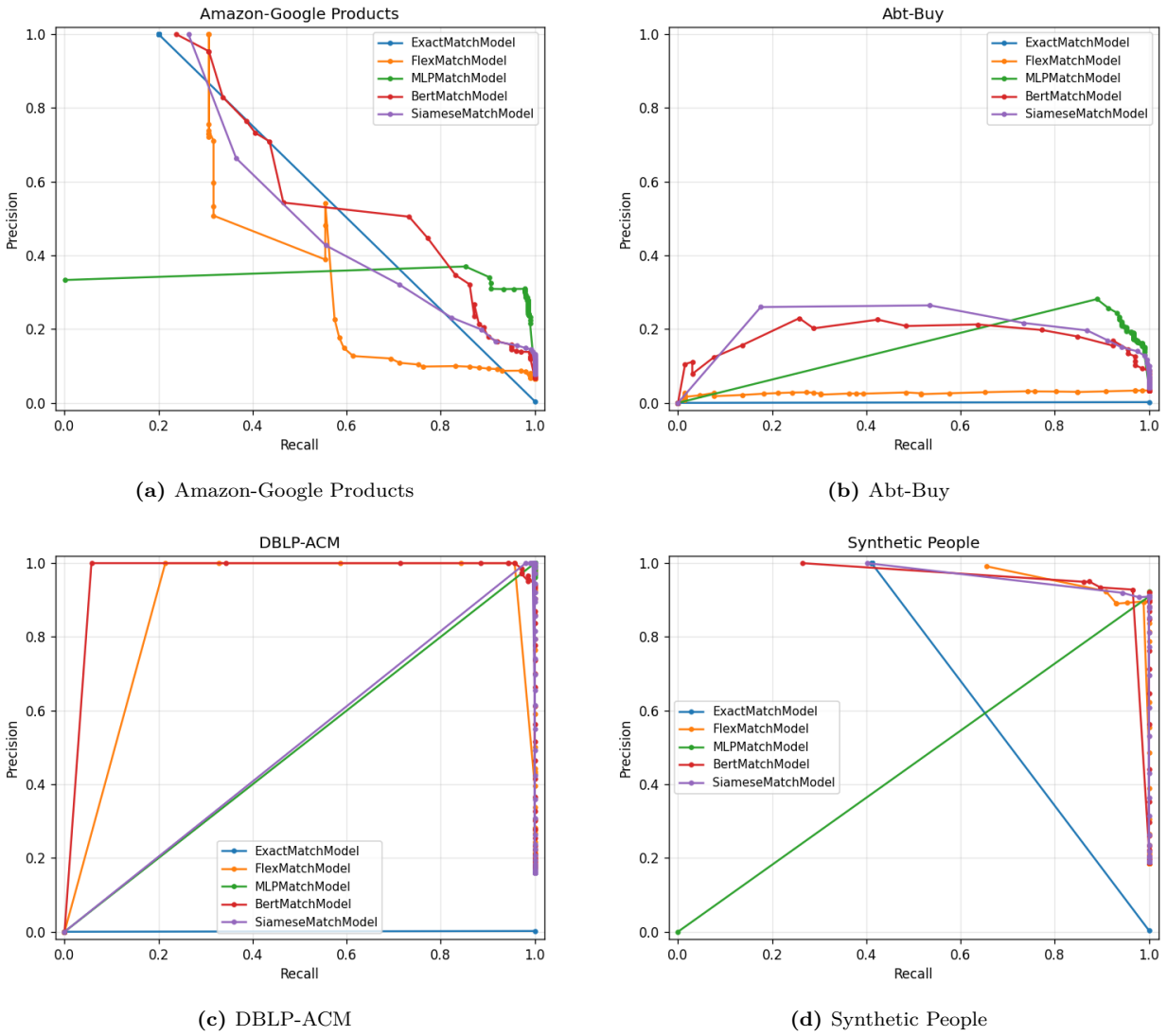
Across the four datasets a single message recurs. The choice of methodological tier matters less than the alignment between the method's inductive bias and the dataset's data-generating process. Pretrained encoders excel when the dataset's matching relation is a form of semantic paraphrase. Engineered feature classifiers excel when the relation is a form of structured noise. Fine-tuning helps in the former case but cannot rescue the latter from a fundamentally mismatched representation.

This finding is consistent with the broader observation in machine learning that strong inductive bias often beats general-purpose representation when the dataset is small and the task is narrow.

8 Comparison to Published Baselines

For ceiling reference we compare the best F1 in our protocol against the F1 reported by two strong deep-ER systems on the same datasets. Table 5 reports these numbers.

DeepMatcher numbers are from Mudgal et al. (2018) Table 4 (Hybrid model). Ditto numbers are from Li



(a) Amazon-Google Products

(b) Abt-Buy

(c) DBLP-ACM

(d) Synthetic People

Figure 1: Precision-recall curves across the four benchmark datasets. Each curve is one model swept across thresholds 0 to 1 in steps of 0.02. ExactMatch collapses to a point because it produces a binary score.

et al. (2020). The Magellan baseline of Konda et al. (2016) is not directly comparable because the underlying `py_entitymatching` library does not install on Python 3.10 or above at the time of writing.

These comparisons require a critical caveat. DeepMatcher and Ditto both report F1 on a labeled-pair test set, that is, a balanced set of positive and negative pairs sampled from the perfect mapping. We report F1 on the all-pairs candidate set produced by blocking, which is much larger and includes many more easy negatives. The two quantities measure different things. The labeled-pair F1 asks "given a pair, is it a match," while the all-pairs F1 asks "of all candidates this model produced, how many of the predicted matches were correct." A meaningful direct comparison would require re-running DeepMatcher and Ditto under our protocol, which we have not done. We treat the cited numbers as a ceiling reference rather than a head-to-head bar.

Within our protocol, DBLP-ACM is essentially saturated by the MLP at F1 1.000. Amazon-Google and Abt-Buy retain headroom, with the gap to the published numbers traceable to a combination of blocking recall, threshold calibration, and the protocol mismatch.

Table 5: F1 reported on these datasets by two strong deep-ER systems, compared to this package's best F1.

Dataset	DeepMatcher	Ditto	This work
Amazon-Google	0.694	0.751	0.431 (MLP)
Abt-Buy	0.628	0.892	0.302 (MLP)
DBLP-ACM	0.984	0.989	1.000 (MLP)

9 Limitations

The benchmark is run on the first 500 records of each dataset rather than the full corpus, in order to keep the wall-clock time within a single-laptop budget of approximately 25 minutes for the full five-models-on-four-datasets sweep. Full-data results would require approximately one to two orders of magnitude more compute and were deferred. The relative ordering across models is unlikely to invert on full data, but absolute MRR and F1 levels would change.

The Magellan baseline was not re-run because `py_entitymatching` does not install on the Python 3.10+ environment in which the rest of the package runs. The DeepMatcher and Ditto numbers cited in Section 8 are from those papers' published evaluations under their own protocols.

The synthetic person dataset was generated by the package itself. While the underlying perturbation rules are documented and reproducible, the resulting corpus is not standardized and its results should be interpreted as a controlled case study rather than a benchmark.

The held-out group split partitions groups 70/30 with a single random seed. A more careful evaluation would average across multiple splits. The current results report multi-seed variance for the stochastic models but not multi-split variance for the deterministic models.

10 Reproducibility

The package, datasets, and trained model artifacts are available at the GitHub repository linked from the Zenodo record. The full results in this paper can be reproduced from a fresh checkout with:

```
git clone https://github.com/johnnysaldana/matchify.git
cd matchify
python3.12 -m venv .venv
source .venv/bin/activate
make install-deep
make bench
```

The `make bench` target runs the full five-models-on-four-datasets sweep with the held-out split, three seeds, and the precision-recall threshold sweep. Output is written to `output.html` for the side-by-side prediction tables and to `docs/pr/` for the precision-recall curves. The benchmark takes approximately 25 minutes on an Apple M-series MacBook.

The package archive is registered with Zenodo for citation. See `CITATION.cff` for the canonical citation.

Acknowledgements

This work was carried out under the supervision of Dr. Tom Lippincott of the Center for Language and Speech Processing at Johns Hopkins University, in the course EN.601.507 Applied Entity Resolution and Deduplication, Spring 2023. The literature reading list and the methodological framing draw on the textbooks of Christen (2012) and Papadakis et al. (2021) and on the papers collected during the course.

References

Zeinab Bahmani, Leopoldo Bertossi, and Nikolaos Vasiloglou. ERBlox: Combining matching dependencies with machine learning for entity resolution. *International Journal of Approximate Reasoning*, 83:118–141, 2017.

Omar Benjelloun, Hector Garcia-Molina, David Menestrina, Qi Su, Steven Euijong Whang, and Jennifer Widom. Swoosh: a generic approach to entity resolution. *The VLDB Journal*, 18(1):255–276, 2009.

Mikhail Bilenko and Raymond J. Mooney. Adaptive duplicate detection using learnable string similarity measures. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2003.

Olivier Binette and Rebecca C. Steorts. (almost) all of entity resolution. *Science Advances*, 8(12), 2022.

Peter Christen. *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Data-Centric Systems and Applications. Springer, 2012.

William W. Cohen, Pradeep Ravikumar, and Stephen E. Fienberg. A comparison of string distance metrics for name-matching tasks. In *Proceedings of the IIWeb Workshop*, 2003.

Muhammad Ebraheem, Saravanan Thirumuruganathan, Shafiq Joty, Mourad Ouzzani, and Nan Tang. Distributed representations of tuples for entity resolution. *Proceedings of the VLDB Endowment*, 11(11): 1454–1467, 2018.

Ivan P. Fellegi and Alan B. Sunter. A theory for record linkage. *Journal of the American Statistical Association*, 64(328):1183–1210, 1969.

Raia Hadsell, Sumit Chopra, and Yann LeCun. Dimensionality reduction by learning an invariant mapping. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2006.

Pradap Konda, Sanjib Das, G. C. Paul Suganthan, AnHai Doan, Adel Ardalan, Jeffrey R. Ballard, Han Li, Fatemah Panahi, Haojun Zhang, Jeffrey Naughton, Shishir Prasad, Ganesh Krishnan, Rohit Deep, and Vijay Raghavendra. Magellan: Toward building entity matching management systems. *Proceedings of the VLDB Endowment*, 9(12):1197–1208, 2016.

Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710, 1966.

Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. Deep entity matching with pre-trained language models. *Proceedings of the VLDB Endowment*, 14(1):50–60, 2020.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. Deep learning for entity matching: A design space exploration. In *Proceedings of the ACM International Conference on Management of Data (SIGMOD)*, 2018.

George Papadakis, Dimitrios Skoutas, Emmanouil Thanos, and Themis Palpanas. *The four generations of entity resolution*. Synthesis Lectures on Data Management. Morgan and Claypool, 2021.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

Jiannan Wang, Tim Kraska, Michael J. Franklin, and Jianhua Feng. CrowdER: Crowdsourcing entity resolution. *Proceedings of the VLDB Endowment*, 5(11):1483–1494, 2012.

William E. Winkler. String comparator metrics and enhanced decision rules in the Fellegi-Sunter model of record linkage. In *Proceedings of the Section on Survey Research Methods, American Statistical Association*, 1990.